Experiment 7

Student Name: Mehak Mehra
UID: 24BAI70075
Branch: CSE - AIML
Section/Group: 24AIT_KRG G1
Semester: 4
Date of Performance: 13/3/26
Subject Name: DBMS
Subject Code: 24CSH-298

Aim

To design and implement a materialised view and to compare and analyse execution time and performance differences between simple views, complex views, and materialized views, thereby understanding their impact on query optimization and system performance.

Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

Objectives

- To create simple views, complex views, and materialized views, and to evaluate their performance by comparing query execution time for each, highlighting the advantages of materialized views in enterprise-level applications.

Practical/Experiment Steps

- Relational Schema Construction: Developed the depts and emps tables, establishing a primary-foreign key relationship to simulate an enterprise organizational structure.

- Simple View Implementation: Created a standard virtual view (V_SIMPLE) to filter specific columns and rows from a single table based on salary thresholds.
- Complex Logic Aggregation: Designed a complex view (V_COMPLEX) that integrates multi-table joins and aggregate functions like SUM() and AVG() for departmental budgeting.
- Materialized Storage Configuration: Implemented a Materialized View (V_MATERIALIZED) to physically store precomputed query results, reducing the overhead of real-time calculations.
- Performance Benchmark Analysis: Utilized the EXPLAIN ANALYZE utility to measure and compare the execution costs and retrieval times across all three view types.
- Data Refresh Synchronization: Executed the REFRESH MATERIALIZED VIEW command to ensure the stored data reflects the most recent updates from the underlying base tables.

Procedure

- Initialized the PostgreSQL environment via pgAdmin and established a connection to the local database server.
- Executed the DDL scripts to create the depts and emps tables and populated them with representative organizational data.
- Defined a Simple View to extract high-salary employee names and verified the output with a basic SELECT statement.
- Constructed a Complex View using an INNER JOIN and GROUP BY clause to calculate total and average salaries by department.
- Created a Materialized View using the same logic as the complex view to demonstrate how results are persisted in storage.
- Simulated a data update scenario and practiced the REFRESH command to synchronize the materialized view with the base tables.
- Conducted an execution time analysis by running EXPLAIN ANALYZE on each view to observe the differences in query planning and execution speed.
- Evaluated the performance metrics, noting the shift from real-time computation in complex views to direct data retrieval in materialized views.
- Saved the execution logs and performance reports to document the efficiency gains achieved through materialized caching.

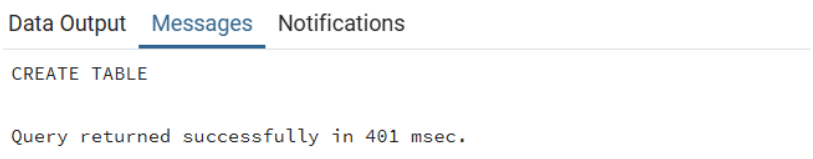
Input/Output Analysis

SQL Input Queries

```
CREATE TABLE depts(
dept_id INT PRIMARY KEY,
dept_name VARCHAR(20)
);

CREATE TABLE emps(
emp_id INT PRIMARY KEY,
name VARCHAR(20),
dept_id INT REFERENCES depts(dept_id),
salary NUMERIC
);
```

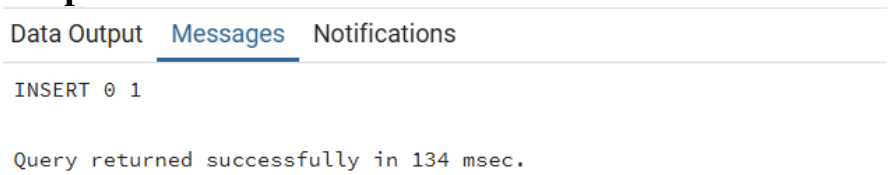
Output



SQL Input Queries

```
INSERT INTO depts VALUES(1, 'IT'), (2, 'HR'), (3, 'SALES');
INSERT INTO emps VALUES(101, 'Mary', 1, 95000);
INSERT INTO emps VALUES(102, 'Amit', 1, 85000);
INSERT INTO emps VALUES(103, 'Sarah', 2, 70000);
INSERT INTO emps VALUES(104, 'John', 2, 65000);
INSERT INTO emps VALUES(105, 'Jack', 3, 55000);
INSERT INTO emps VALUES(106, 'Rohan', 1, 88000);
```

Output

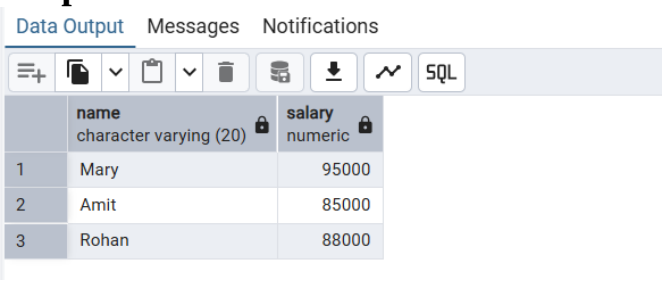


SQL Input Queries

```
CREATE VIEW V_SIMPLE AS
SELECT name, salary FROM emps WHERE salary>75000;

SELECT * FROM V_SIMPLE;
```

Output



Data Output | Messages | Notifications

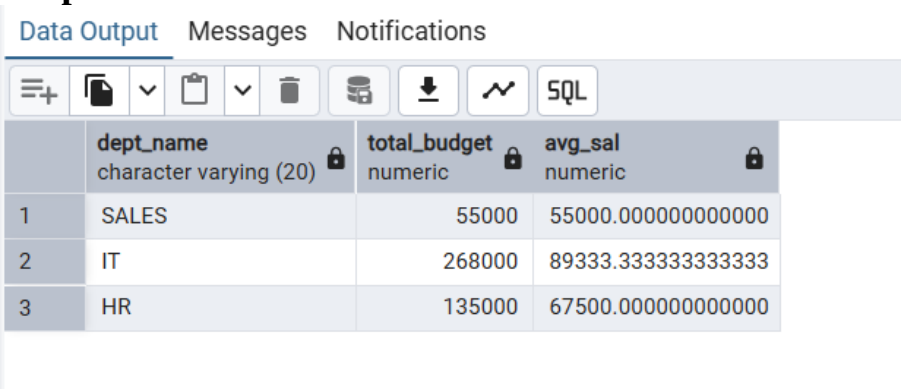
	name character varying (20)	salary numeric
1	Mary	95000
2	Amit	85000
3	Rohan	88000

SQL Input Queries

```
CREATE VIEW V_COMPLEX AS
SELECT d.dept_name, SUM(e.salary) AS total_budget, AVG(e.salary) AS avg_sal
FROM emps e JOIN depts d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM V_COMPLEX;
```

Output



Data Output | Messages | Notifications

	dept_name character varying (20)	total_budget numeric	avg_sal numeric
1	SALES	55000	55000.000000000000
2	IT	268000	89333.333333333333
3	HR	135000	67500.000000000000

SQL Input Queries

```
CREATE MATERIALIZED VIEW V_MATERIALIZED AS
SELECT d.dept_name, SUM(e.salary) AS total_budget, AVG(e.salary) AS avg_sal
FROM emps e JOIN depts d
ON e.dept_id = d.dept_id
```

GROUP BY d.dept_name;

SELECT * FROM V_MATERIALIZED;

Output

Data Output | Messages | Notifications

	dept_name character varying (20)	total_budget numeric	avg_sal numeric
1	SALES	55000	55000.000000000000
2	IT	268000	89333.333333333333
3	HR	135000	67500.000000000000

SQL Input Queries

REFRESH MATERIALIZED VIEW V_MATERIALIZED;

Output

Data Output | Messages | Notifications

```
REFRESH MATERIALIZED VIEW

Query returned successfully in 175 msec.
```

SQL Input Queries

EXPLAIN ANALYZE SELECT * FROM V_SIMPLE;

Output

Data Output | Messages | Notifications

	QUERY PLAN text
1	Seq Scan on emps (cost=0.00..18.00 rows=213 width=90) (actual time=0.018..0.021 rows=3.00 loop...
2	Filter: (salary > '75000'::numeric)
3	Rows Removed by Filter: 3
4	Buffers: shared hit=1
5	Planning Time: 0.101 ms
6	Execution Time: 0.033 ms

SQL Input Queries

EXPLAIN ANALYZE SELECT * FROM V_COMPLEX;

Output

Data Output | Messages | Notifications

Showing rows: 1 to 16

	QUERY PLAN text
1	HashAggregate (cost=51.54..54.54 rows=200 width=122) (actual time=0.110..0.117 rows=3.00 loops=1)
2	Group Key: d.dept_name
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=2
5	-> Hash Join (cost=30.25..48.34 rows=640 width=90) (actual time=0.081..0.089 rows=6.00 loops=1)
6	Hash Cond: (e.dept_id = d.dept_id)
7	Buffers: shared hit=2
8	-> Seq Scan on emps e (cost=0.00..16.40 rows=640 width=36) (actual time=0.036..0.037 rows=6.00 loops=1)
9	Buffers: shared hit=1
10	-> Hash (cost=19.00..19.00 rows=900 width=62) (actual time=0.036..0.036 rows=3.00 loops=1)
11	Buckets: 1024 Batches: 1 Memory Usage: 9kB
12	Buffers: shared hit=1
13	-> Seq Scan on depts d (cost=0.00..19.00 rows=900 width=62) (actual time=0.025..0.027 rows=3.00 loo…
14	Buffers: shared hit=1
15	Planning Time: 0.558 ms
16	Execution Time: 0.184 ms

SQL Input Queries

EXPLAIN ANALYZE SELECT * FROM V_MATERIALIZED;

Output

Data Output | Messages | Notifications

	QUERY PLAN text
1	Seq Scan on v_materialized (cost=0.00..15.40 rows=540 width=122) (actual time=0.026..0.027 rows=3.00 loo…
2	Buffers: shared hit=1
3	Planning Time: 0.090 ms
4	Execution Time: 0.041 ms

Learning Outcomes

- Gained proficiency in differentiating between virtual simple/complex views and physically stored materialized views.
- Gained the ability to use EXPLAIN ANALYZE to interpret query plans and identify performance bottlenecks.
- Learned the lifecycle of materialized views, including creation, storage benefits, and manual refresh mechanisms.
- Understanding how precomputing results in materialized views supports high-performance requirements for companies like SanDisk and PayPal.